
UNIVERSIDADE FEDERAL DO AMAZONAS

Programa de Pós-Graduação em Engenharia Elétrica — PPGE

Linha 1 — Sistemas Inteligentes e Microeletrônica

Critical Summary

“A Survey on Automated Symbolic Verification and its Application for Synthesizing Cyber-Physical Systems”

L. C. Cordeiro, E. B. de Lima Filho, I. V. Bessa

IET Cyber-Physical Systems: Theory & Applications, 2020

Disciplina: PGENE601 — Verificação e Síntese Automática de
Sist. Ciber-Físicos e Embarcados

Docente: Prof. Dr. Lucas C. Cordeiro

Discente: Matheus Serrão Uchôa

Nível: Pós-Graduação — Mestrado

Data: Manaus, September 16, 2026

Reference

Full reference. L. C. Cordeiro, E. B. de Lima Filho, I. V. Bessa. “A Survey on Automated Symbolic Verification and its Application for Synthesizing Cyber-Physical Systems.” *IET Cyber-Physical Systems: Theory & Applications*, vol. 5, no. 1, pp. 1–24, 2020. DOI: 10.1049/iet-cps.2018.5006.

1. Objectives

The survey addresses the growing reliability demands placed on *embedded and cyber-physical systems* (ECPS) — systems that combine discrete computation with continuous physical dynamics and are now pervasive in automotive, medical, industrial, and IoT applications. The authors set three explicit goals: (i) briefly describe the state of the art in ECPS modeling, verification, and synthesis; (ii) provide a technical overview of *symbolic* verification methods, with particular emphasis on bounded model checking (BMC) and its extensions, which the authors identify as their own “vision for future ECPS verification and synthesis schemes”; and (iii) present open challenges in ECPS verification and synthesis and clarify current trends, which the authors argue are largely driven by ECPS growth, the Internet of Things, and machine learning (ML). The survey’s scope is explicitly restricted to *automated symbolic* verification and synthesis; simulation-based and purely statistical approaches are mentioned only for contrast, not reviewed in depth.

2. Methods

Two distinct notions of “method” operate in this paper: the survey methodology itself, and the technical verification/synthesis methods it surveys.

2.1 Survey methodology

The authors conducted a systematic literature search across three digital libraries — Google Scholar, Web of Science, and Scopus — following a three-stage protocol of *searching*, *screening*, and *synthesis*. To mitigate bias inherent to automated database queries, search results were cross-validated against a manually curated “control list” of papers from peer-reviewed venues already known, *a priori*, to be relevant to symbolic verification and synthesis for ECPS. Screening was guided by inclusion criteria centered on the core topic keywords (e.g., “Internet-of-Things”, “Concurrent Systems”, “Hybrid Systems”, “Control Systems”). The authors candidly acknowledge, in a dedicated limitations section, that

this screening stage remains partly subjective and that their search may not cover the entire ECPS literature.

2.2 Technical methods surveyed

ECPS are first formalized as *hybrid automata*: a tuple $S = (X, X_0, U, Y, r, v)$ of reachable states, initial states, inputs, outputs, a transition relation, and an output map. This modeling layer is illustrated throughout the paper with a running example, the *Sim3Tanks* three-tank system, whose safety and liveness requirements are expressed both in computation tree logic (CTL) and in linear-time temporal logic (LTL). Building on this foundation, the survey reviews six families of symbolic **verification** technique:

1. **Bounded Model Checking (BMC)**. Unrolls a transition system to a fixed depth k and checks satisfiability of the resulting verification condition via SAT or SMT solvers. Sound, but — absent a known completeness threshold — inherently incomplete.
2. **k -induction**. Strengthens BMC into a complete proof technique by combining a base case with an inductive step over automatically inferred loop invariants, removing the need for users to manually supply them.
3. **Property-Directed Reachability (IC3/PDR)**. An alternative inductive-invariant search that is guided by counterexamples-to-induction (CTIs); the survey reports that IC3 scales better than k -induction on certain benchmark classes.
4. **Craig interpolation**. Derives an over-approximation (an interpolant) of reachable states directly from an SMT solver’s unsatisfiability proof, offering a route to invariant generation that complements k -induction and IC3.
5. **Abstraction**. Predicate abstraction with counterexample-guided refinement (CEGAR) and abstract interpretation reduce model size by tracking only relevant predicates or numerical intervals, trading precision for scalability (representative tools: SLAM, SATABS, Astrée, PolySpace).
6. **Path-based symbolic execution**. Explores individual program paths (e.g., KLEE), useful for coverage-driven test generation but subject to path explosion on large programs.

For **synthesis**, the survey centers on *Counter-Example Guided Inductive Synthesis* (CEGIS): an iterative SYNTHESIZE–VERIFY loop that refines a candidate controller

against a correctness specification using counterexamples as new constraints, until a verified solution is found or the search fails. Its application to ECPS is illustrated through **DSVerifier/DSSynth**, an ESBMC-based front-end that instruments digital-controller code (direct, delta, and cascade realizations) with finite-word-length semantics and checks overflow, limit-cycle, timing, stability, and minimum-phase properties — the same tool examined earlier in this course’s exercise list on DSVerifier and k -induction, which gives this summary a direct, concrete anchor to the practical material already covered.

3. Results (Key Findings)

As a survey, this paper’s “results” are synthesized findings rather than experimental data:

- **Eleven concrete challenges** for embedded/ECPS verification and synthesis are catalogued: time and energy constraints; handling concurrent software; platform restrictions; security issues; verifying code modules embedded in larger structures; legacy designs; support for multiple programming languages and interfaces; correct code instrumentation; non-linear/non-convex optimization; incorporation of new checks as systems evolve; and provision for system evolution over successive versions.
- These challenges are distilled into **seven open Research Problems (RP1–RP7)**: suitable SMT encodings for realistic ECPS background theories; scaling BMC-based verification of multi-threaded software; proving both correctness *and* timeliness via stronger k -induction/invariant techniques; verifying system-level (not merely code-level) properties; broadening tool, language, and application support; more robust automated formal synthesis; and simultaneous verification of low- and high-level properties within a single framework.
- A cross-reference table explicitly links each research problem to the challenges it addresses, showing that the most demanding problems (RP1 and RP7) cut across nearly every challenge category identified.
- A dedicated section looks **beyond code correctness**, discussing SMT-based non-convex optimization (via a CEGIS variant, CEGIO) and introducing *antifragility*: systems designed not merely to resist faults but to strengthen from exposure to them — an idea the authors connect to a future combination of ML with symbolic verification loops.

- The authors explicitly report their own **limitations**: bias inherent to automated digital-library search, a screening stage acknowledged as partly subjective, and a scope restricted to symbolic (as opposed to statistical or purely simulation-based) methods.

4. Conclusion

The survey concludes that, while BMC, k -induction, IC3, Craig interpolation, abstraction, and path-based symbolic execution are individually mature techniques, there remains limited published evidence of their systematic *combination* (e.g., k -induction strengthened with learned invariants, or IC3 combined with induction) applied to ECPS specifically. Formal synthesis via CEGIS, in turn, is shown to be fundamentally dependent on — and bottlenecked by — the soundness of the underlying verification models it queries for counterexamples. The authors’ own future-work agenda mirrors this gap directly: a *unified* framework that jointly addresses code verification, behavior correctness, environment adaptation, and system evolution, with machine learning incorporated to guide search and strengthen invariants. This positions the survey less as a finished map of the field and more as an explicit research agenda — one that the tools examined earlier in this course (ESBMC, DSVerifier, and the k -induction algorithm) already partially instantiate, but that the authors argue is still far from the “unified, high- and low-level, correct-by-construction” vision they set out for ECPS.